

Architecture

Top-Level Structure

·	
├─ backend/	Flask API, stats helpers, CSV data, source JSON
├─ frontend/	React/Vite app
├─ old-website-backup/	Previous static HTML/CSS/JS version
├─ .github/workflows/	GitHub Pages deployment workflow
├─ Dockerfile	Container build for frontend + backend
├─ render.yaml	Render API service config
├─ railway.json	Railway Docker builder config
└─ start.sh	Container startup script

Runtime Shape

The frontend and backend are separate pieces:

1. User opens the React app.
2. React loads public assets such as background images, the CTC logo, and music files.
3. React calls the backend API using `VITE_API_URL` if set, otherwise the hardcoded fallback `https://ctcmkwiistatswebsite.onrender.com`.
4. Flask receives `/api/...` requests.
5. Flask reads the matching CSV file from `backend/CSV`.
6. Helper functions in `stats.py` and `find.py` calculate averages and rankings.
7. Flask returns JSON arrays or objects to the frontend.

Important Entry Points

- Frontend app: `frontend/src/App.tsx`
- Frontend routes/components:
 - `frontend/src/pages/HomePage.tsx`
 - `frontend/src/components/PlayerStats.tsx`
 - `frontend/src/components/TopTeamPlayers.tsx`
 - `frontend/src/components/TopTracks.tsx`
 - `frontend/src/components/BestMatchups.tsx`
- Backend web app: `backend/app.py`
- Backend stats helpers: `backend/stats.py`
- CSV lookup helpers: `backend/find.py`
- JSON-to-CSV extraction script: `backend/extract.py`

`backend/main.py` is not the web app. It imports modules and calls `stats.findplayeravg("zilla")`, so it appears to be a scratch/test script.

Data Model

The active API reads flattened CSV rows:

```
team,player,track,score  
6c,brody,Bowser Jr.'s Fort,12
```

Every race result is one row. A 5v5 match with 12 races can create up to 120 player-race rows.

Division Selection

The frontend sends a **division** query parameter:

- 1_2
- 3
- 4

The backend maps that directly to a CSV path:

```
backend/CSV/ctc_d{division}.csv
```

Examples:

- **division=1_2** -> **backend/CSV/ctc_d1_2.csv**
- **division=3** -> **backend/CSV/ctc_d3.csv**
- **division=4** -> **backend/CSV/ctc_d4.csv**

This means future seasons cannot be added cleanly through the current API unless the data naming and query model are expanded.

Current Limitations

- Season is not a first-class parameter.
- CSV files are the API source of truth.
- Uploading a new match file does not automatically rebuild analytics.
- Some API responses are formatted strings instead of structured JSON.
- Caching is in-memory per Flask process and has no invalidation endpoint.
- The frontend has repeated **API_URL** constants in several files.